`/aida-plan` (+ `docs/plans/`) vs `/ultraplan`

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09 — incorporates details from a vendor-published overview of Ultraplan's research-preview workflow (browser review surface, "teleport back to terminal" handoff, GitHub-repo requirement, Remote Control incompatibility). /ultraplan terms (3 free uses, then Max-quota or billed; 90-minute approval window before the cloud session terminates) captured live from the launch prompt 2026-05-13 and may shift while the feature is in research preview. Re-verify against code.claude.com/docs/en/claude-code-on-the-web before relying on the cost line.

The TL;DR: **`/ultraplan` drafts and reviews the prose in the browser. AIDA persists, structures, verifies, and executes the result. The clean integration point is `/ultraplan`'s own "teleport back to terminal → save plan to file" option, which lands the plan exactly where AIDA's `docs/plans/` convention wants it.**

This is the sister doc to vs-ultrareview.md. The same thesis applies twice: the Claude Code cloud `/ultra*` family is great at LLM-heavy generation and browser-native review; AIDA owns the surrounding agent-collaboration layer (graph, IDs, traces, queue, persistence, MCP). The skeptic's question — *"I use /ultraplan and it's great, why do I need AIDA?"* — has a sharper answer once you've used `/ultraplan` twice and lost the chat history both times.

Different scopes, complementary

	`/aida-plan` + `docs/plans/`	`/ultraplan`
Engine	Local Claude session orchestrating req decomposition + plan file authoring	Cloud-based LLM generating dense, file-by-file plan documents
Cost	$0 — uses the active Claude session	3 free uses then Max quota or billed per use (research-preview; verify current terms); 90-minute approval window before timeout
Trigger	`/aida-plan <SPEC>` in any Claude session	`/ultraplan <prompt>`, typing "ultraplan" anywhere in a prompt, or refining a local plan in cloud

	`/aida-plan` + `docs/plans/`	`/ultraplan`
Output shape	Child requirement decomposition + design-decision comments + optional plan file under `docs/plans/`	Single dense markdown document: approach, file-by-file, risks, verification script, followups
Review surface	Edit the file, use git diff, comment on the parent req	Browser: inline comments on passages, emoji reactions (approve/revise), structured outline sidebar, iterative comment-and-revise cycle with Claude
Status indicators	`aida list --status planned` + plan file in git	CLI status indicator: Claude researching / needs clarification / ◆ `ultraplan ready`
Anchoring	Symbol refs preferred (`aida plan verify [--fix]`, TASK-93 shipped — re-anchors stale refs); plan stored as a git-tracked file linked in the req graph	Line refs (drift fast — 2 of 8 line refs were already stale on day-of-generation in the STORY-86 case study)
Persistence	First-class file in repo, linked from STORY/EPIC, queueable	Browser session evaporates after 90 min; the “save plan to file” teleport-back option preserves it — but only if you remember to choose that option
Execution path	Plan + queue routing + implementer session (local)	Two choices after approval: (1) cloud execution — Claude implements in same web session, opens PR from browser; (2) “teleport back to terminal” — plan returns local
Integration	Plans become part of git history, referenced by Related Requirements, surfaced in session manifests (planned TASK-95)	Standalone unless you teleport-back-and-save; the saved file is then an inert artifact unless AIDA picks it up
Iteration	Edit the plan file, recommit, diff is visible	Browser review-revise loop is genuinely good; once teleported back, iteration is git-based
Offline	Works fully offline	Requires cloud round-trip + Claude Code on the web account + GitHub-connected repo
Driven by	The requirement graph — knows children, parents, sibling specs, trace links	The diff between current code and the user’s prompt — no req-graph awareness
Launchable from	Anywhere a Claude session can run	User-triggered only (Claude Code controls); approval window forces human-in-the-loop; can’t run simultaneously with Claude Code Remote Control

---

What `/ultraplan` does that bare AIDA can't (today)

The STORY-86 case study (`docs/plans/2026-05-13-story-86-done-status.md` — saved from a real `/ultraplan` cloud session) demonstrates the genuine depth `/ultraplan` brings:

Strength	What it looked like in the STORY-86 plan
Executive-summary discipline	One paragraph stating the whole strategy before any details — readable in 30 seconds
ASCII state-machine + control-flow diagrams	The Draft → … → Done → Completed lifecycle and the auto-bump helper's control flow rendered in ~10 lines of art
File-by-file changes with line refs	~25 files enumerated, each with the specific edit shape
"Critical Files" enumeration	Must-touch paths surfaced separately from prose — at-a-glance blast radius
"Reusable helpers — do not reimplement"	Explicit list (`extract_spec_ids_from_commit`, `record_role_activity`, `Storage::update_atomically`) so the implementer doesn't re-invent parsers
Risks + gotchas with mitigations	8 numbered: serde downgrade compat, force-push rewrites, first-pull-on-new-clone, etc.
Decision callouts	"Where does X live? Both. Here's why." — resolved decisions, not open-question lists
Complexity estimate	LOC + commits + risk level — sets the implementer's clock
Test cases by name	Specific `fn` names, not "add tests"
Verification script	~30-line end-to-end bash smoke (positive + negative) — definition of done as executable
Followups (defer)	Explicit "file these at completion time" list — lighter than child reqs, no scope creep
Order-matters annotation	"Do edits top-to-bottom so each commit builds clean"

AIDA's `/aida-plan` skill is more decomposition-oriented (vertical-slice child reqs, design-decision comments) and doesn't generate the dense file-by-file prose. That's a real gap when the work warrants it.

Multi-agent architecture (reported, hedged)

A third-party explainer describes `/ultraplan`'s internal architecture as a Mixture-of-Agents pattern: three parallel "explorer" agents each independently attempt the

planning problem in separate context windows, then a fourth "critic" agent evaluates their outputs and synthesizes a final plan that may include elements from none of the explorers verbatim. This claim is plausible (it matches published MoA research and would explain the consistently high "decision callout" quality we observed in the STORY-86 plan) but should be attributed to a secondhand explainer rather than vendor-confirmed documentation. Treat the structural details as best-effort context, not specification.

If the description is accurate, the practical implications are:

- **Genuine diversity, not paraphrased sameness.** Independent context windows let explorers arrive at substantively different approaches (architectural pattern, tradeoff profile, abstraction level, risk tolerance) rather than three rewrites of the same idea.
- **Critic synthesis beats single-explorer best.** The critic can combine the architecture from explorer 1 with the error-handling from explorer 2 and the test strategy from explorer 3 — final plan quality exceeds any single run.
- **Convergence-as-confidence signal.** When all three explorers converge on a similar approach, that's strong evidence the approach is sound. When they diverge, the critic surfaces explicit tradeoff discussion. A single-agent plan can't tell you whether the AI was confident or just committed to its first thought.
- **Wall-clock parallelism.** Three explorers running in parallel costs roughly the time of one explorer running alone, plus the critic pass. Higher quality without proportional latency.
- **Multi-agent invoked conditionally.** Simple requests reportedly bypass the full pipeline — single-agent suffices. This explains why some /ultraplan outputs feel denser than others.

AIDA's contrast: `/aida-plan` runs in a single local Claude session — same context window throughout, no parallel exploration, no separate critic. The local model can be prompted to "consider three approaches, then evaluate," but anchoring bias persists because all three approaches share the same context. This is a real asymmetry; AIDA shouldn't claim parity here.

AIDA does apply the same insight elsewhere: the implementer/reviewer split in AIDA's session model is the critic-must-be-separate principle internalized — a reviewer Claude session never shares context with the implementer Claude session whose work it's reviewing, precisely to prevent the anchoring bias that single-context self-review would suffer. See `docs/session-lifecycle.md` "Why roles don't share Claude sessions" for how that architectural commitment shapes AIDA's role boundaries and the (future) TUI orchestration vision.

Browser review surface — separate from the prose strength

Distinct from the depth of the generated plan itself, `/ultraplan` brings a review-and-revise interface that local-terminal planning genuinely can't match:

- **Inline comments on specific passages** — feedback is targeted to the exact paragraph or section, not a general response to a 300-line document.
- **Emoji reactions** — quick approve/revise signals on sections, lower friction than typing.
- **Structured outline sidebar** — jump between sections without scrolling; useful for long plans (the STORY-86 plan was 11 sections, ~300 lines).

- **Iterative cycle** — comment, Claude revises, present updated draft, repeat until ready. The browser holds the conversation thread visibly; the terminal would require scrolling chat history.
- **Status indicator in CLI** — ◆ `ultraplan ready` and equivalents for researching / needs-clarification, so the terminal stays free while the cloud session works.

These are not things AIDA replicates. They're real strengths in the specific niche of "review a planning artifact before any code changes." A team that's coordinating a complex migration and wants multiple reviewers leaving inline comments on the plan has no equivalent local-tooling option.

What AIDA does that `/ultraplan` doesn't

The skeptic's argument starts to crack here. `/ultraplan`'s output, sitting alone in a chat window, has no answer to any of these:

- **Persistence by default, not by remembering.** `/ultraplan` does offer a "save to file" option in its teleport-back menu — but only if you remember to choose it before the 90-minute approval window closes. Two of the three teleport-back options (inject into session, start new session) discard the plan as soon as the chat ends. AIDA plans live in `docs/plans/YYYY-MM-DD-<slug>.md` from the moment they're saved, git-tracked, referenced by `Related Requirements` — no "remember to click the right option" failure mode.
- **Graph membership.** AIDA plans are pinned to their target SPEC-ID via `aida comment add`, surfaceable from `aida show STORY-86`, queryable through the MCP server. `/ultraplan` doesn't know what your SPEC IDs are.
- **Stable identifiers.** AIDA's symbol refs survive edits; `/ultraplan`'s line refs go stale within a day. (STORY-86 plan: 2 of 8 refs drifted within hours. The `DbCommand::Sync` callsite ref was off by ~19,000 lines.)
- **Verifiability.** `aida plan verify [--fix]` (TASK-93, shipped) re-anchors stale line refs, validates file paths, and lints structural sections — no equivalent in `/ultraplan`'s output.
- **Queue + session integration.** `aida queue work <SPEC>` rides the matching `docs/plans/` file's Critical-Files/Followups/Verification brief into a fresh implementer session (TASK-95, shipped; `/aida-pickup` leads with it) — the plan is loaded as context, not something to grep for.
- **Followups auto-extraction.** Reaching Done/Completed parses the plan's `## Followups` section and offers to file each as a child TASK, idempotent via an `[aida:followups]` marker (TASK-96, shipped). `/ultraplan`'s followups list is inert markdown.
- **Cross-vendor exposure.** AIDA plans live in the git-canonical store — reachable via the token-efficient CLI or the MCP server, and now genuinely multi-vendor-readable (Codex is a first-class vendor alongside Claude), so a saved plan is cross-vendor-durable. `/ultraplan`'s output is a single-chat, Claude-only artifact.
- **Cost stability.** `/ultraplan`'s terms shifted within months of launch (initially-free → 3 free uses then Max-quota or billed). AIDA's local-first cost stays $0 regardless of vendor pricing changes.

- **Independence from approval windows.** A 90-minute timeout on a planning artifact is a real workflow constraint; missed it once already on STORY-86. AIDA plans never expire.
- **Works without internet.** Field-work, transit, flaky connections — `/ultraplan` is unreachable; AIDA plans aren't.

The complementary workflow — teleport back is the clean handoff

`/ultraplan`'s "teleport back to terminal" menu has three sub-options after the browser approval:

1. **Inject the plan into the current conversation and continue from there.**
2. **Start a new session with the plan as the only context.**
3. **Cancel and save the plan to a file for later review.**

Option 3 is the AIDA-aligned path. The plan file lands in your local environment exactly where AIDA's `docs/plans/YYYY-MM-DD-<slug>.md` convention expects it. Options 1 and 2 are the "no AIDA in the loop" paths — they work, but they re-create the chat-only persistence problem AIDA solves.

For **complex/risky work** where the dense brief is worth the cloud round-trip:

1. **Run `/ultraplan`** with the target spec ID + acceptance criteria. Pay the cloud round-trip for the file-by-file depth.
2. **Review in the browser.** Use the inline comments, emoji reactions, and outline sidebar to revise the plan iteratively with Claude until ready. (This is the part AIDA can't replicate; lean into it.)
3. **Teleport back, choose "save to file."** Plan lands locally. Move/rename to `docs/plans/YYYY-MM-DD-<slug>.md` (AIDA convention). Commit it.
4. **`aida comment add <SPEC-ID>`** with a one-liner pointing at the plan file. The plan becomes graph-reachable.
5. **`aida queue add <SPEC-ID> --for implementer`** routes the work.
6. **`aida queue work <SPEC-ID>`** launches a fresh implementer session that opens with the plan's brief as context (TASK-95).
7. **Implementer executes**; `aida plan verify <file>` (TASK-93) re-anchors any stale line refs as the code shifts under the plan.
8. **Reaching Done/Completed** parses the plan's Followups section and offers to file each as a child TASK under the parent (TASK-96) — no out-of-scope items get forgotten.

For **routine work** (small feature, well-bounded bug fix):

1. **`/aida-plan`** alone — decompose into child reqs, comment design decisions on the parent. The decomposition often *is* the plan. No need to spend a `/ultraplan` use on it.
2. **Skip the cloud round-trip.** The session that has full context already has enough to execute.

The empirical rule: if the file-by-file list would be longer than ~15 entries AND multiple reviewers need to leave comments on the plan, `/ultraplan` + teleport-back-to-file earns its keep. Below that, AIDA-only is faster.

When cloud execution makes sense (and when it skips AIDA)

`/ultraplan`'s alternative path is **cloud execution** — after browser approval, Claude implements the plan in the same web session, presents a diff view, and opens a PR all from the browser. The terminal isn't involved.

This is genuinely useful for:

- Work on a forge-hosted project with no local development dependencies
- Reviewers who want to approve-and-merge without round-tripping to a developer's machine
- Quick refactors that need no local integration testing

It is NOT compatible with the AIDA model because:

- No local implementer session means no `// trace:<SPEC>` comments are added by an agent that knows the requirement graph
- No `aida queue done` to flip status and surface followups
- No `aida edit --status completed` integration; the PR merging is the only signal AIDA could pick up on (which is what TASK-83 / EPIC-21 wire eventually, but it's after-the-fact reconstruction rather than first-class tracking)
- No spec-anchored execution rationale; the PR commit messages may or may not reference SPEC-IDs

If your work needs AIDA's lifecycle tracking, **always teleport back**. If your work is genuinely a one-off where the bookkeeping doesn't pay back, cloud execution is fine.

Tightening AIDA's integration with `/ultraplan`

`/ultraplan` exposes no API or MCP surface (as of research preview, 2026-05-13). Integration is **workflow-tight, not API-tight.** Two real directions:

Direction A — AIDA → `/ultraplan`: rich prompt assembly

Today the user types a free-form prompt. AIDA has rich context for any SPEC: description + acceptance criteria + comments (including design seeds) + parent/child/sibling reqs + trace-comment graph + plan-template scaffolding. `aida ultraplan <SPEC>` (TASK-113, shipped) assembles all of this into a structured prompt that gives `/ultraplan`'s three explorers something concrete to anchor on — turning a 100-word user prompt into a ~2000-token prompt with the full requirement graph behind it.

Shape:

```
aida ultraplan <SPEC>           # assemble rich prompt, copy to clipboard
aida ultraplan <SPEC> --stdout  # print for inspection / piping
aida ultraplan <SPEC> --json   # prompt + warnings + token estimate, for scripting
```

The assembled prompt includes: the target SPEC's description and extracted `## Acceptance` criteria, parent/child/sibling spec summaries (siblings capped to fit the token budget), the AIDA 11-section plan structure (from TASK-92) inlined so the returned plan matches `docs/plans/_TEMPLATE.md`, the trace-graph reusable helpers (from TASK-94's `build_reusable_helpers_section`), and a "symbol refs preferred" style note. It copies to the clipboard by default and falls back to stdout when no clipboard tool is available. There is no `--browser` mode: `/ultraplan` is an in-session Claude Code trigger, not a URL-launched surface.

Direction B — `/ultraplan` → AIDA: auto-import saved plan

After the user teleports back and saves the plan file, the `/aida-import-plan` skill (TASK-114, shipped) processes it into AIDA's first-class state in one command:

1. Detect target SPEC-ID (from filename, frontmatter, or first heading)
2. Rename + move to `docs/plans/YYYY-MM-DD-<slug>.md` (AIDA convention)
3. Post `aida comment add <SPEC>` pinning the plan to its target
4. Parse Critical Files section into the session manifest hint (composes with TASK-95)
5. Parse Followups section into draft TASKs (composes with TASK-96)
6. Run `aida plan verify <file>` to re-anchor line refs to symbols (composes with TASK-93)
7. Optionally `aida queue add <SPEC> --for implementer` if invoked with `--queue`

What's NOT possible today

- **Programmatic invocation** of `/ultraplan` from AIDA — no API/MCP exposed
- **Reading `/ultraplan` output without manual teleport-back-save** — browser session is opaque from outside
- **Capturing the 3-explorer outputs separately** to show divergence — only the critic's synthesized output is exposed
- **Watching the CLI status indicator** (◆ `ultraplan ready`) programmatically — no documented hook

If Anthropic eventually exposes an MCP server for `/ultraplan`, the integration could become much tighter (e.g., AIDA polling for "ready" status, auto-pulling the output, programmatic confidence-signal extraction). Until then, workflow-tight is the ceiling.

Research preview + requirements + limitations

`/ultraplan` is currently a **research preview**, which has practical implications:

- **Behavior may change** between releases. Workflows pinned to specific UI details (status indicator glyphs, teleport-back menu options) may need re-verification.
- **Cost terms shifted** within months of launch — initially marketed as Max-included, the 2026-05-13 launch prompt observation showed 3 free uses then quota/billed. Future shifts likely.
- **Requirements**: a **GitHub repository** AND a **Claude Code on the web** account. GitLab / Bitbucket / self-hosted-git users are excluded.

- **Incompatibility**: cannot run simultaneously with Claude Code's **Remote Control** feature. Both share the `claude.ai/code` interface; launching `/ultraplan` disconnects an active Remote Control session.

AIDA has none of these constraints — local-first, vendor-neutral, works against any git remote (or no remote), no subscription required. That's not a "win" so much as a different design space: research-preview tooling buys you depth at the cost of dependencies; durable-infrastructure tooling buys you stability at the cost of cloud cycles. Use both where their strengths apply.

What if the skeptic skips AIDA entirely?

The honest answer: **possible, but lossy.**

A team using only `/ultraplan` + `/ultrareview` + manual git workflow can ship code. What they lose:

- **No queryable record** of why a piece of work was undertaken once the chat is gone. Tomorrow's "what was this commit's spec?" has no answer.
- **No spec-to-code traceback.** `git blame` shows who and when; it doesn't show *why* — no `// trace:STORY-86` comment, no `aida show STORY-86` to read the rationale.
- **Brittle lifecycle bookkeeping.** Plans go stale, sessions terminate, status fields drift. `/aida-pickup → /aida-pr → /aida-review`'s atomic close-out doesn't exist.
- **No agent collaboration layer.** A second agent in a separate session has no way to discover what the first agent decided. `/ultraplan`'s output is a single-chat artifact; AIDA's graph is a shared workspace.
- **No MCP exposure.** Editor-resident agents (Cursor, Continue, Aider, IDE Claude extensions) can't see the planning trail.

This is genuinely the trade-off. If your project is a 2-week prototype with one developer and zero handoffs, the loss is mostly hypothetical. If it's a 6-month codebase with multiple contributors (human or agent) and the question *"why did we do X this way?"* will come up later — AIDA's defensible niche **is** that the answer survives.

Honest scope statement

`/aida-plan`'s value **isn't** *"better plans than `/ultraplan`."* `/ultraplan`'s LLM-heavy cloud cycles produce denser, more thorough planning artifacts than a single local Claude session can match. The defensible thing AIDA brings is *"plans that persist, integrate with the requirement graph, and stay anchored as the code evolves."* That's a complementary capability, not a substitute.

If a team has to pick one of the two for budget reasons, the right answer depends on what they're optimizing for:

- *"Generate the most thorough plan per work item"* → `/ultraplan`. Cloud LLM cycles dominate single-session planning on raw output density.

- *"Keep planning artifacts queryable, versioned, and integrated with the code over time"* → AIDA. No competitor in this slot.
- *"Plan without a vendor subscription or internet"* → AIDA. Local-first always works.

Both fit in most workflows. The point of this doc is that picking *one* in a binary way is almost always wrong — and the cost of compose-them is one file save + one comment.

See also

- vs-ultrareview.md — sister positioning doc, same thesis applied to code review
- STORY-112 — Plan-mode skill placeholder ("inspired by /ultraplan"); parent of the plan-tooling roadmap
- TASK-92 — Structured plan template (the 11-section convention extracted from /ultraplan dissection)
- TASK-93 — `aida plan verify` (re-anchor stale line refs to symbols)
- TASK-94 — Auto-derive "Reusable helpers" section from trace graph
- TASK-95 — `aida queue work` pre-populates session manifest from matching plan file
- TASK-96 — `aida queue done` extracts Followups section, offers to file as TASKs
- `docs/plans/2026-05-13-story-86-done-status.md` — worked example: the actual `/ultraplan` output that prompted this doc
- composition.md — generic guidance on layering AIDA with other tools (future page)